JAVA 2

Module COL

Collections

Au programme

1 Une structure qui grandit

2 Les listes

3 Les ensembles

4 Les tables associatives

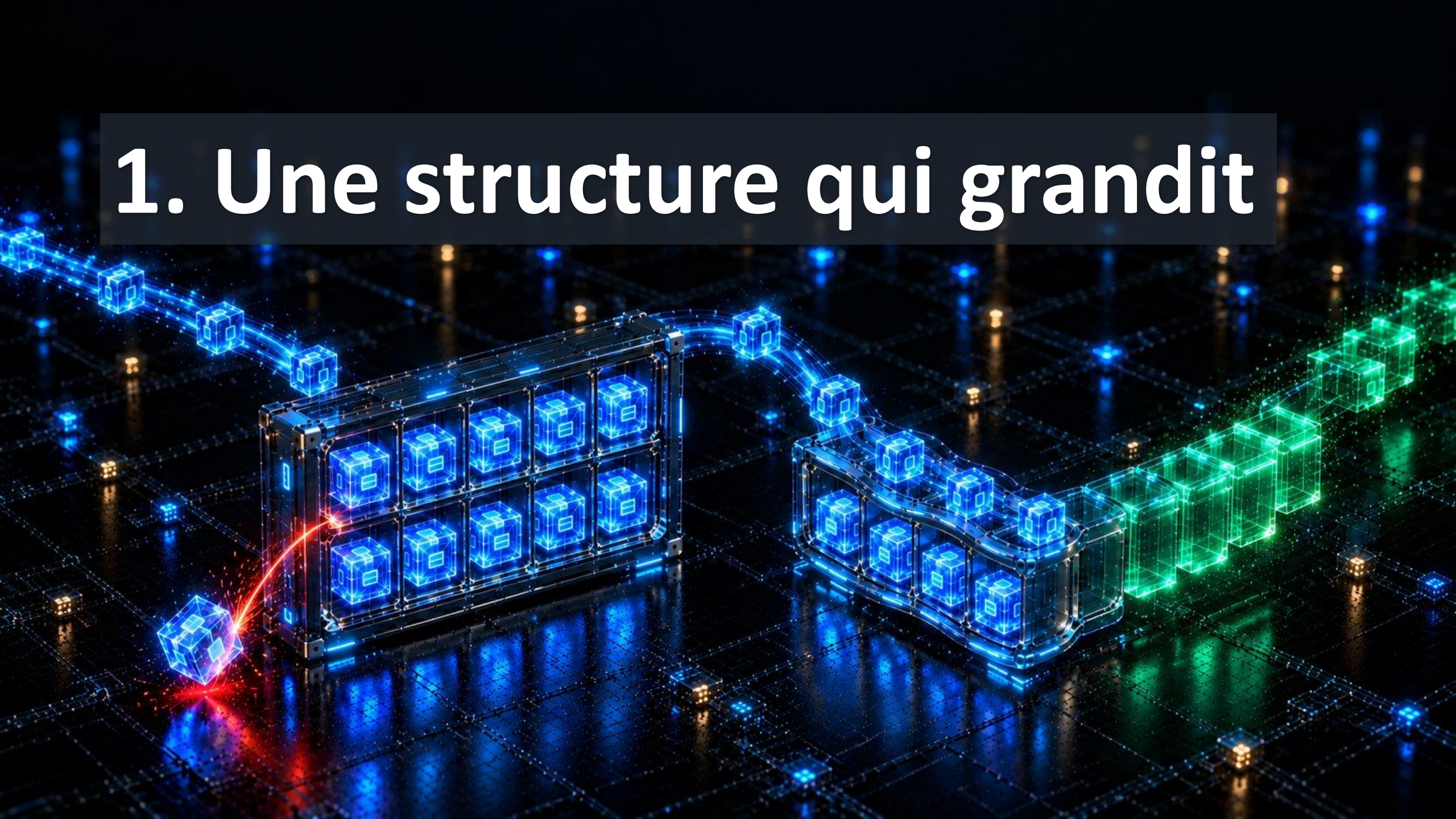
5 Passer un comportement

6 Comparer et trier

7 Collections emboîtées

8 Les files à deux bouts

1. Une structure qui grandit



Démonstration



Démonstration

Choisir une collection en se trompant d'abord

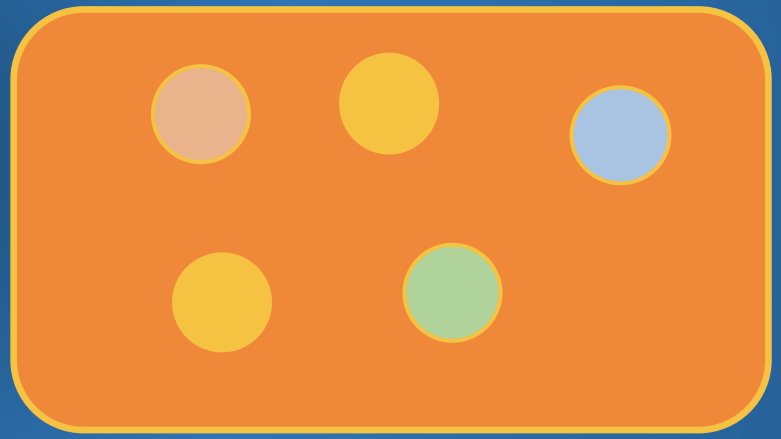
Déroulé : Demo_COL

Une collection

Des objets rassemblés dans une seule structure

Elle grandit et rétrécit au fil des ajouts

Chaque sorte ajoute sa promesse : un ordre, l'unicité, une clé, deux bouts



Ce qu'on écrit entre chevrons

Le type entre chevrons (< et >) est celui des éléments de la collection

Le compilateur refuse tout autre type à l'entrée

Et il connaît le type de ce qui sort

```
List<String> titres = new ArrayList<>();

titres.add("Germinal");
titres.add(42);                    // refuse a la compilation

String premier = titres.get(0);  // aucune conversion
```

Des objets, pas des types primitifs

Une collection ne contient que des objets

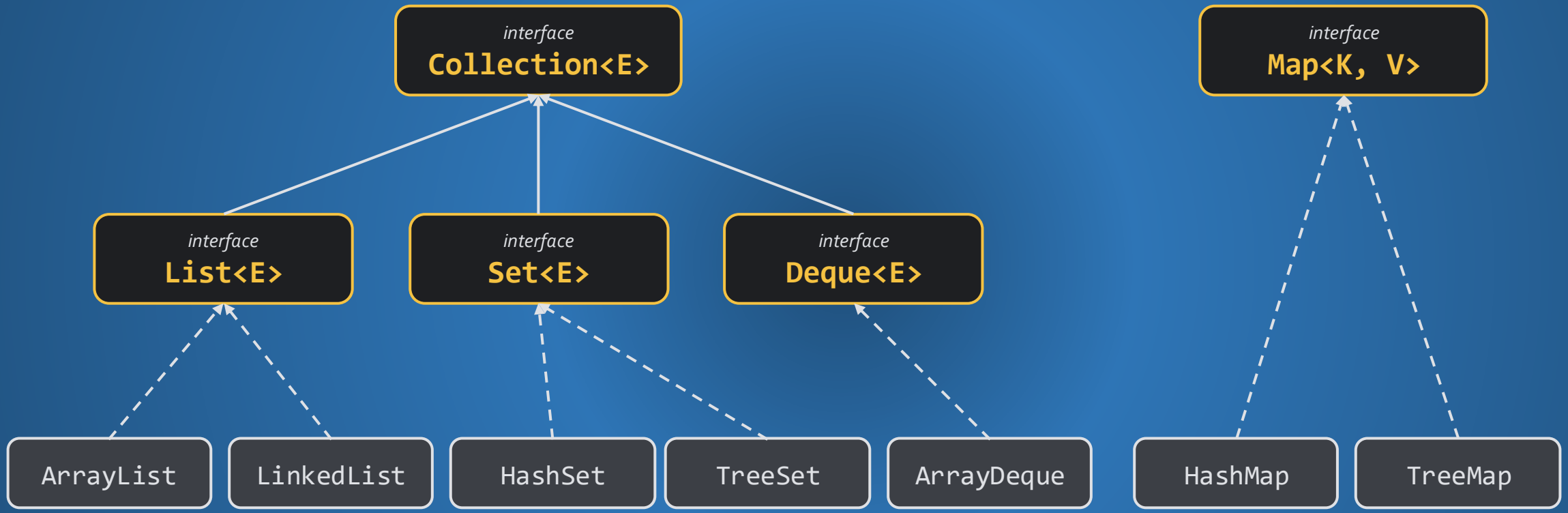
On écrit donc `Integer` et non `int`

La conversion reste automatique, dans les deux sens

```
List<Integer> annees = new ArrayList<>();   // List<int> : refuse

annees.add(1885);                  // int vers Integer
int premiere = annees.get(0);      // Integer vers int
```

La famille des collections



trait plein : une interface en étend une autre ; pointillé : une classe implémente l'interface ; `Map` n'est pas une Collection

Déclarer par l'interface

À gauche, l'interface : ce que la variable promet

À droite, la classe : comment la promesse est tenue

Changer d'implémentation ne change qu'un mot

```
List<String> titres = new ArrayList<>();
Set<String> auteurs = new HashSet<>();
Map<String, Integer> pages = new HashMap<>();
```

2. Une liste, des positions

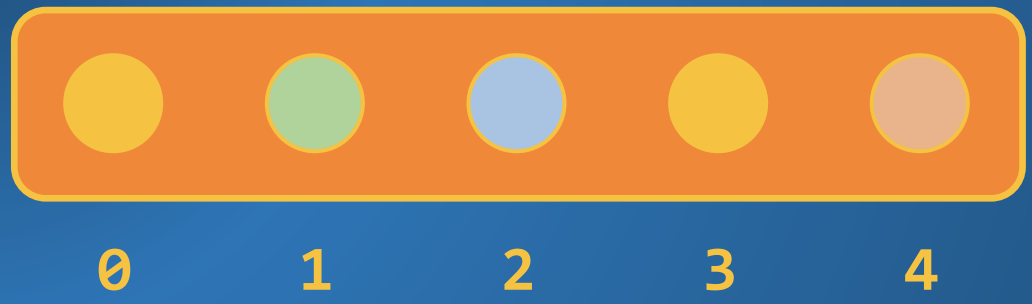


Une liste

Une collection ordonnée : chaque élément a une position

Les positions sont numérotées à partir de o

La même valeur peut figurer plusieurs fois



Les opérations d'une liste

Une liste garde l'ordre d'ajout, et accepte les doublons

Chaque élément a une position, qui commence à o

```
List<String> titres = new ArrayList<>();
titres.add("Germinal");
titres.add("Nana");
titres.add(0, "Candide");          // insere en tete

titres.size();                     // 3
titres.get(1);                     // "Germinal"
titres.contains("Nana");           // true
titres.remove("Nana");             // retire l'element
```

Parcourir une collection

La boucle `for` des tableaux parcourt toute collection

Une collection s'affiche lisiblement, contrairement à un tableau

```
for (String titre : titres) {
    System.out.println(titre);
}

System.out.println(titres);      // [Candide, Germinal]
```

Ce que la boucle `for-each` utilise

Une collection est `Iterable` : elle fournit un `Iterator`

L'itérateur répond à `hasNext()` et `next()`

La boucle `for-each` l'écrit pour vous

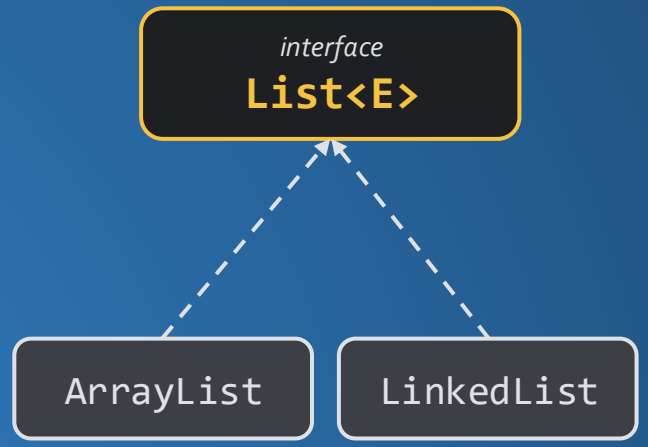
```
Iterator<String> it = titres.iterator();
while (it.hasNext()) {
    String titre = it.next();
    System.out.println(titre);
}
```

`ArrayList` ou `LinkedList`

Deux classes, la même interface `List`

La classe `ArrayList` est le choix par défaut

La classe `LinkedList` est aussi une file à deux bouts



`Collection` et `Collections`

L'interface `Collection` : ce que toutes les collections savent faire

La classe `Collections` : des méthodes statiques qui travaillent dessus

Ces méthodes reçoivent souvent une `List`

```
Collections.sort(titres);              // trie la liste elle-meme
Collections.shuffle(titres);           // la melange
String premier = Collections.min(titres);
int fois = Collections.frequency(titres, "Nana");
```

Nommer ses bornes

Un nombre écrit en clair ne dit pas d'où il vient

Une constante nommée sert au calcul et au message

Le message ne peut plus mentir : CC-NOM.7

```
private static final int MINIMUM = 0;
private static final int MAXIMUM = 1000;

System.out.printf("Nombres entre %d et %d%n", MINIMUM, MAXIMUM);
int tire = MINIMUM + hasard.nextInt(MAXIMUM - MINIMUM + 1);
```

Exercices



COL.1 Deux façons d'être trié

fascicule COL

COL.3 Lire des noms et les trier

fascicule COL

Nommer : CC-NOM.7

3. Un ensemble, sans doublons

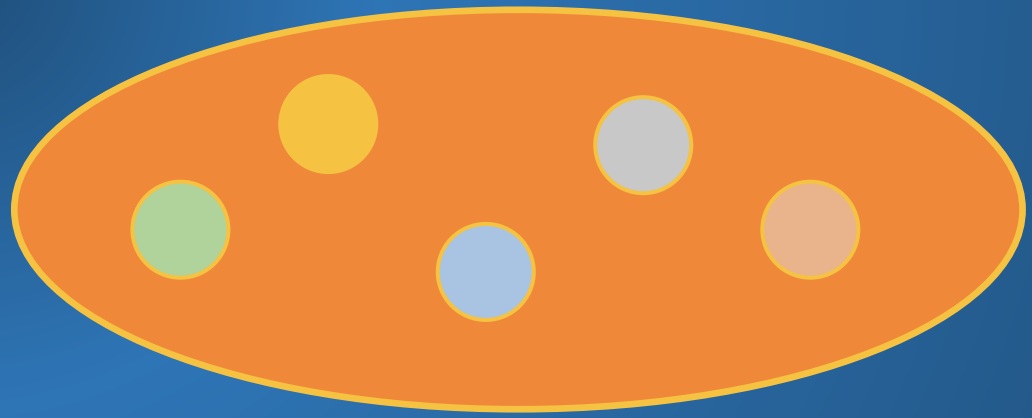


Un ensemble

Chaque valeur n'y figure qu'une fois

Pas de position : on demande si une valeur y est

Le choix quand l'unicité est la règle



Ce qu'un ensemble refuse

Un ensemble ne garde qu'un exemplaire de chaque valeur

La méthode `add` retourne `false` si la valeur y est déjà

Pas de position : ni `get(i)`, ni `add(i, x)`

```
Set<String> auteurs = new HashSet<>();

auteurs.add("Zola");              // true
auteurs.add("Zola");              // false : deja present
auteurs.size();                   // 1
auteurs.contains("Zola");         // true
```

Qui décide que deux éléments sont égaux

L'ensemble interroge `equals` et `hashCode`

Un record les écrit pour vous

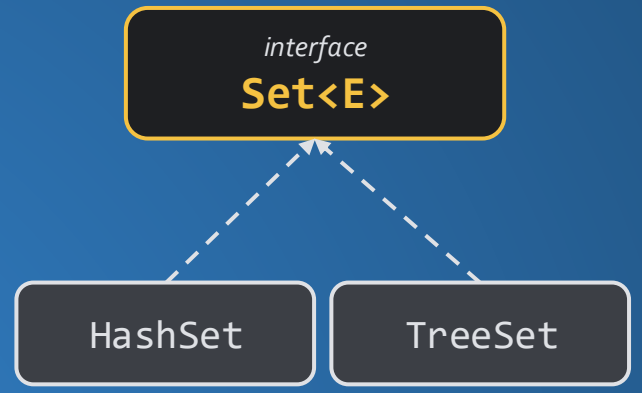
Une classe ordinaire compare des références

```
record Exemplaire(String titre, int numero) { }

Set<Exemplaire> rayon = new HashSet<>();
rayon.add(new Exemplaire("Nana", 1));
rayon.add(new Exemplaire("Nana", 1));

rayon.size();                          // 1
```

Hash ou Tree



	`HashSet`, `HashMap`	`TreeSet`, `TreeMap`
Ordre de parcours	aucun ordre garanti	trié en permanence
Les éléments doivent	définir `equals` et `hashCode`	savoir se comparer
Quand le choisir	par défaut	quand l'ordre compte

Exercices



COL.6 Un ensemble sans doublons

fascicule COL

Nommer : CC-NOM.7, encore

4. Une table, des clés

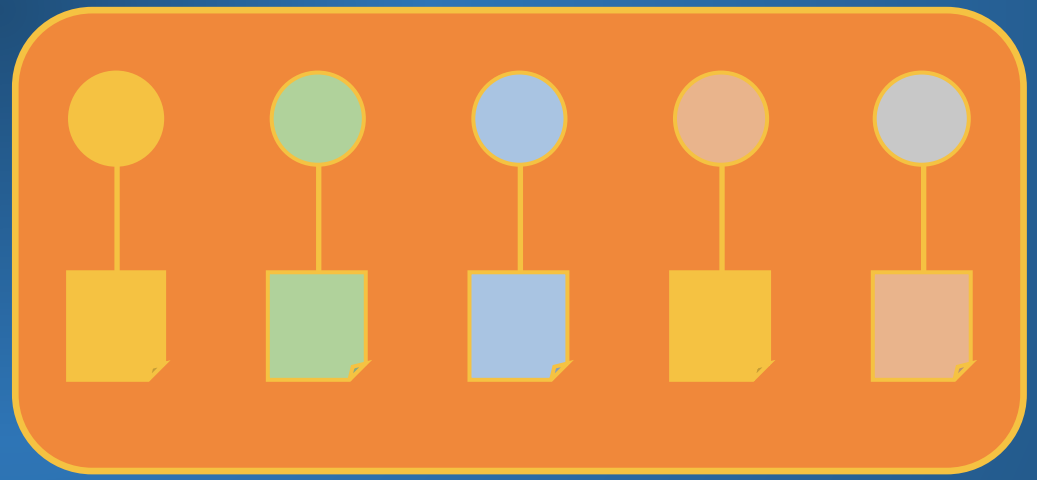


Une table associative

À chaque clé correspond une valeur

On donne la clé, la table retourne la valeur

Les clés sont uniques, les valeurs non



Ranger et retrouver par clé

Une table associative associe une valeur à chaque clé

Une clé n'y figure qu'une fois

Deux types entre chevrons : celui des clés, celui des valeurs

```
Map<String, Integer> pages = new HashMap<>();
pages.put("Germinal", 591);
pages.put("Candide", 160);

pages.get("Candide");              // 160
pages.containsKey("Nana");         // false
pages.size();                      // 2
```

Une clé absente

Pour une clé absente, la méthode `get` retourne `null`

La méthode `getOrDefault` retourne la valeur prévue

```
Integer inconnu = pages.get("Nana");           // null
int nombre = pages.getOrDefault("Nana", 0);    // 0

int plantage = pages.get("Nana");    // NullPointerException
```

Trois façons de parcourir une table associative

Les clés, les valeurs, ou les couples

Chacune est une collection, que la boucle `for` parcourt

```
for (String titre : pages.keySet()) { ... }

for (int nombre : pages.values()) { ... }

for (Map.Entry<String, Integer> entree : pages.entrySet()) {
    System.out.println(entree.getKey() + " : " + entree.getValue());
}
```

Exercices



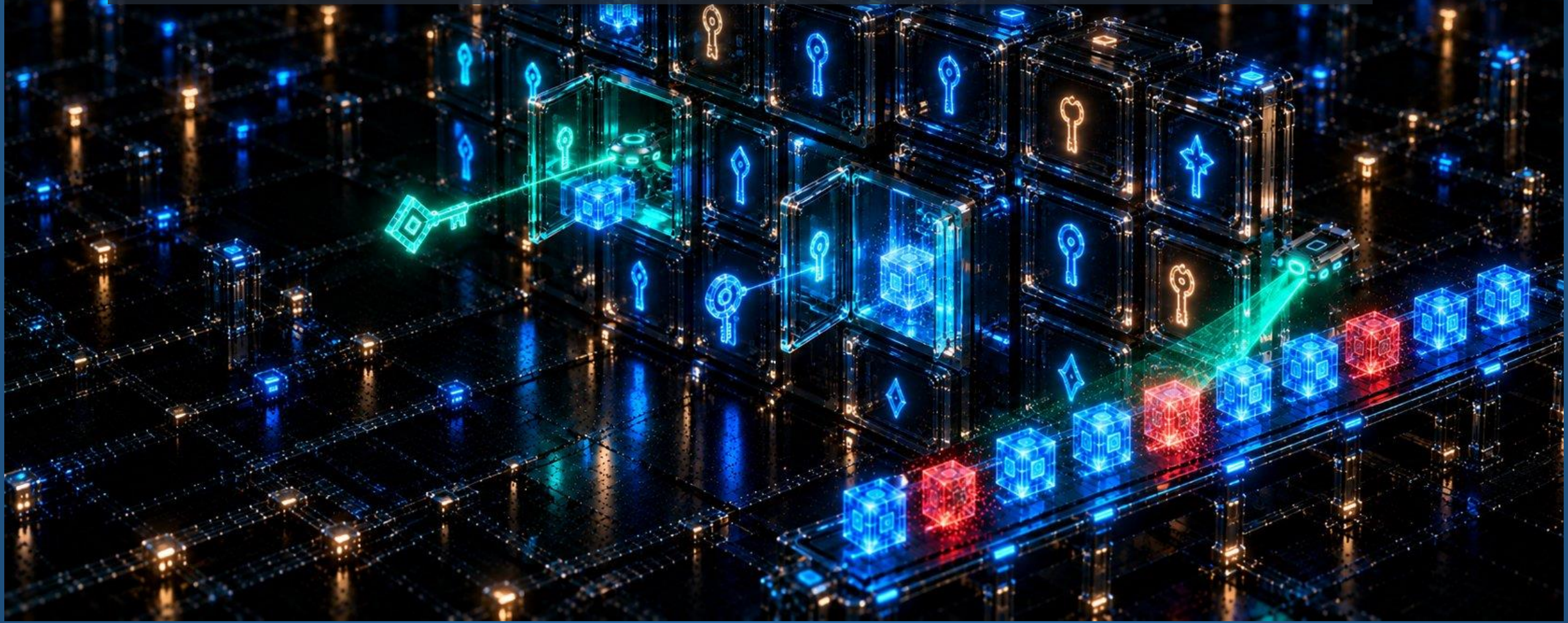
COL.2 Un annuaire

fascicule COL

COL.4 Compter les mots d'un texte

fascicule COL

5. Passer un comportement



Des méthodes ajoutées aux interfaces

Java 8 a ajouté des méthodes par défaut à `Collection` et `Map`

Toutes les classes existantes les ont reçues sans rien changer

Parmi elles : `getOrDefault`, `removeIf`, `computeIfAbsent`

```
// dans l'interface Map<K, V>
default V getOrDefault(Object cle, V parDefaut) { ... }
```

Retirer pendant un parcours de collection

Retirer dans une boucle `for-each` : `ConcurrentModificationException`

L'itérateur, lui, sait retirer sans se perdre

```
for (String titre : titres) {
    titres.remove(titre);       // plantage
}

Iterator<String> it = titres.iterator();
while (it.hasNext()) {
    if (it.next().length() > 20) {
        it.remove();            // sans risque
    }
}
```

Une méthode qui reçoit un critère

La méthode `removeIf` retire ce que le critère accepte

Le critère est un `Predicate`, une interface à une méthode

```
// dans l'interface Collection<E>
default boolean removeIf(Predicate<? super E> filtre) { ... }

// dans l'interface Predicate<T>
boolean test(T valeur);
```

Écrire le critère dans une classe

Une classe implémente `Predicate`

Un objet de cette classe se passe à la méthode `removeIf`

```
public class TitreLong implements Predicate<String> {

    private static final int LONGUEUR_MAX = 20;

    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
}

titres.removeIf(new TitreLong());
```

Exercices

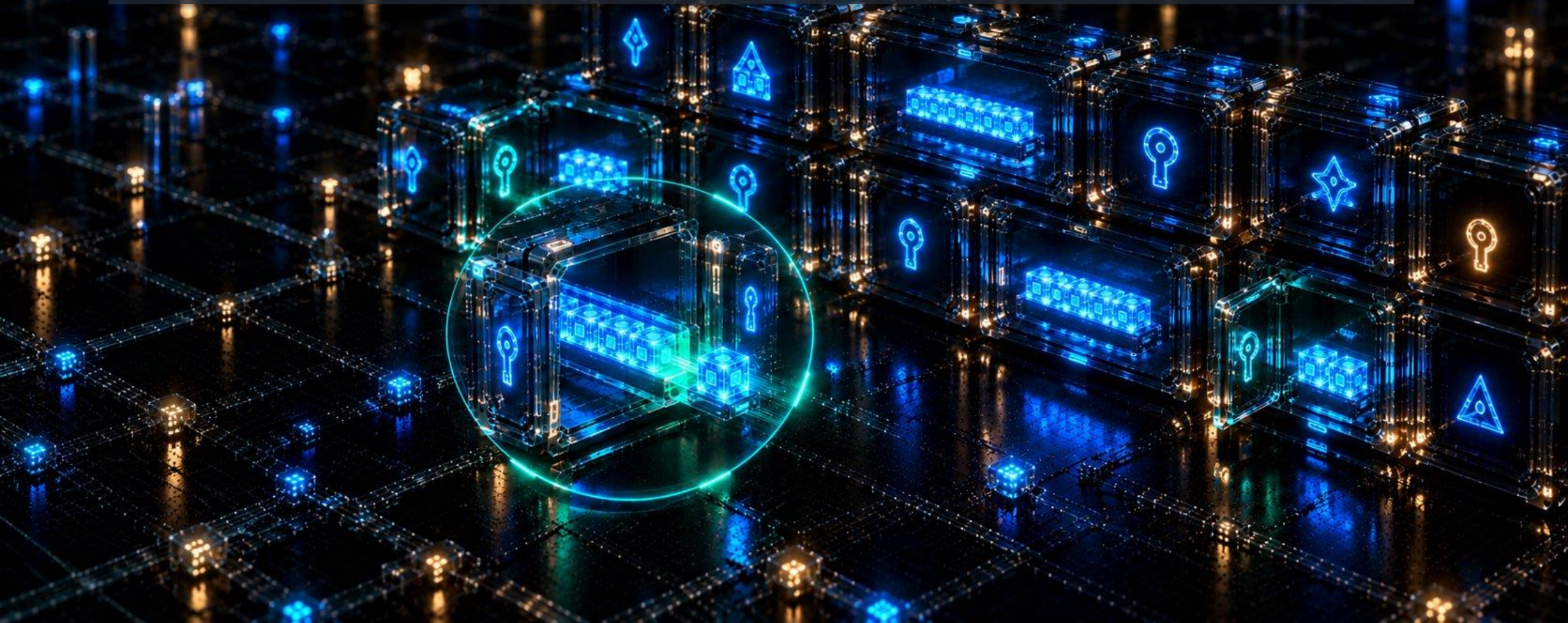


COL.5 Filtrer avec un prédicat

fascicule COL

Nommer : CC-NOM.7, une troisième fois

6. Comparer et trier



Apprendre à un objet à se comparer à un autre

La classe implémente `Comparable`

`compareTo` se lit par son signe : avant, même rang, après

C'est l'ordre naturel, celui d'un `TreeSet`

```
record Exemplaire(String titre, int numero)
        implements Comparable<Exemplaire> {

    @Override
    public int compareTo(Exemplaire autre) {
        return Integer.compare(numero, autre.numero);
    }
}
```

Un ordre fourni de l'extérieur

Un `Comparator` est un objet à part, qui compare deux éléments

On le passe à `sort`, ou au constructeur d'un `TreeSet`

```
public class ParLongueur implements Comparator<String> {
    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
}

titres.sort(new ParLongueur());
Set<String> rayon = new TreeSet<>(new ParLongueur());
```

`Comparable` ou `Comparator` ?

`Comparable` : l'ordre évident, toujours le même

`Comparator` : un autre ordre, plusieurs ordres

`Comparator` : une classe qu'on ne peut pas modifier

Exercices

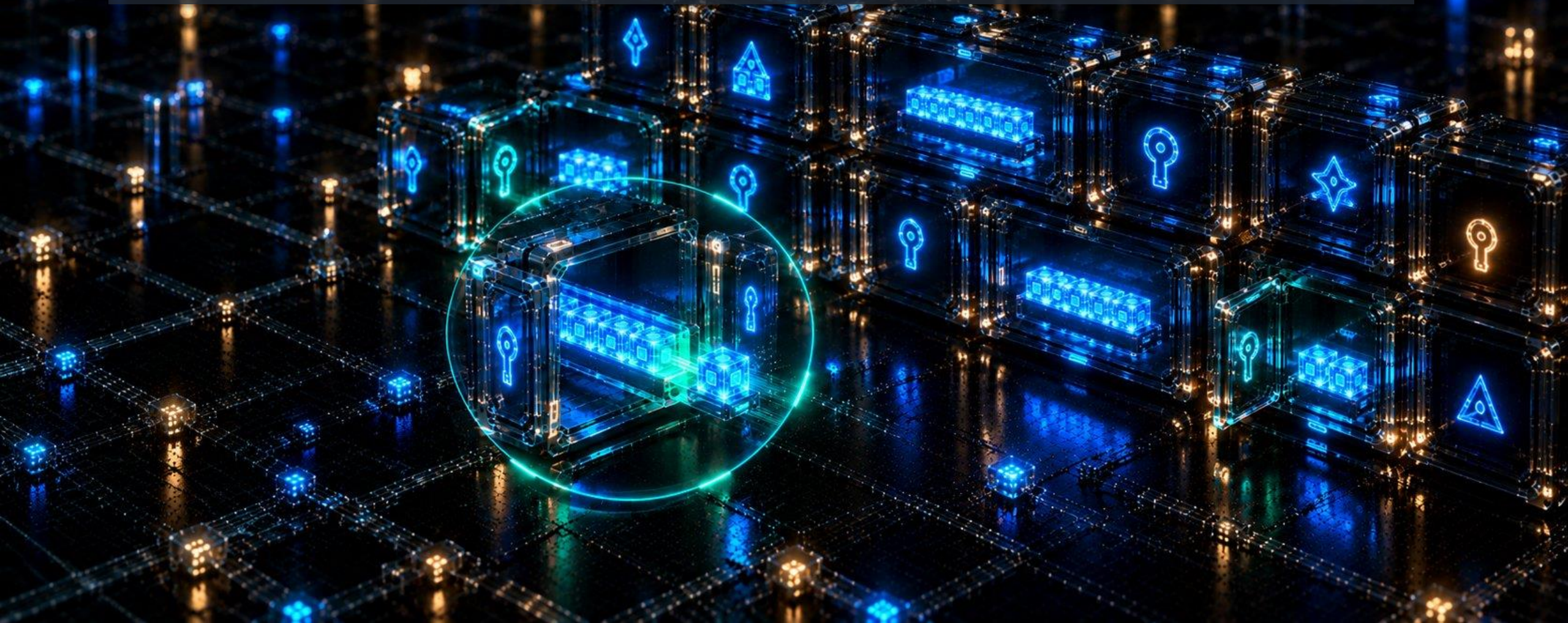


COL.8 Trois ordres pour une bibliothèque

fascicule COL

Départager : `Integer.compare`, jamais une soustraction

7. Des collections emboîtées



Lire un type emboîté

Une table dont chaque valeur est une liste

Le type se lit de l'extérieur vers l'intérieur

La liste d'un auteur n'existe pas avant qu'on la crée

```
Map<String, List<String>> titresParAuteur = new HashMap<>();

List<String> deZola = titresParAuteur.get("Zola");    // null
```

Créer au premier usage

La méthode `computeIfAbsent` retourne la valeur d'une clé

Si elle manque, une `Function` la fabrique d'abord

```
public class NouvelleListe implements Function<String, List<String>> {

    @Override
    public List<String> apply(String auteur) {
        return new ArrayList<>();
    }
}

titresParAuteur.computeIfAbsent("Zola", new NouvelleListe())
               .add("Germinal");
```

Exercices

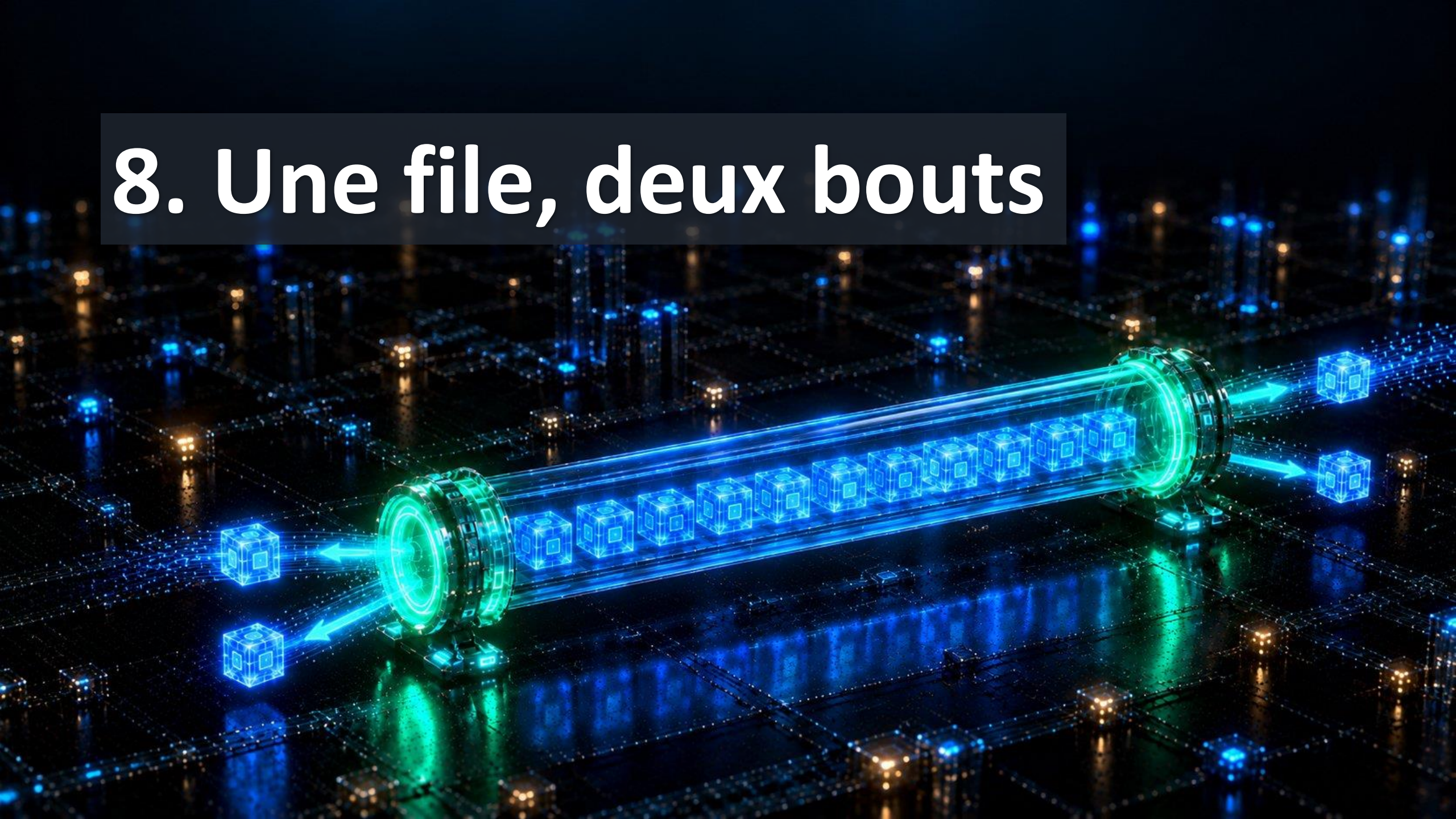


COL.7 Une table de collections

fascicule COL

Retourner : CC-ERR.2 et CC-OBJ.2

8. Une file, deux bouts

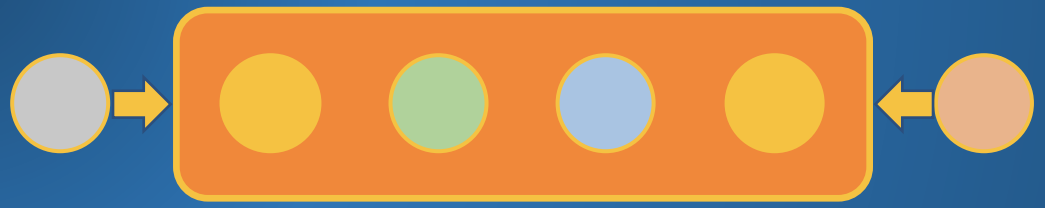


Une file à deux bouts

On n'y touche que par les extrémités

Ajouter ou retirer au début, ou à la fin

Une file d'attente, une pioche de cartes



Ajouter aux deux bouts

On ajoute et on retire aux deux extrémités, jamais au milieu

L'interface `Deque`, et sa classe `ArrayDeque`

Une file d'attente, une pioche de cartes

```
Deque<String> attente = new ArrayDeque<>();

attente.addLast("Zola");
attente.addLast("Hugo");
attente.addFirst("Sand");        // passe devant

System.out.println(attente);     // [Sand, Zola, Hugo]
```

Prendre et remettre

La méthode `pollFirst` retire et retourne le premier élément

La méthode `peekFirst` le retourne sans le retirer

Sur une file vide, les deux retournent `null`

```
String suivant = attente.pollFirst();   // "Sand", retire
String ensuite = attente.peekFirst();   // "Zola", reste

while (!attente.isEmpty()) {
    System.out.println(attente.pollFirst());
}
```

Le fil rouge



G.21 Le deck en `List`
fil rouge GameHub

G.22 La carte et sa valeur
fil rouge GameHub

G.23 La version simplifiée
fil rouge GameHub

Le `Deck` d'`ENU.3` quitte son tableau pour une liste

Le fil rouge, suite



G.24 La méthode jouer, mise en place
fil rouge GameHub

G.25 La méthode jouer, la boucle
fil rouge GameHub

G.26 La méthode jouer, le résultat
fil rouge GameHub

G.27 La vraie bataille
fil rouge GameHub